IsaBegov Hammam Hotel Service Tracker

Project Report

CS308 Software Engineering

Tarik Spahic, 220302224

Mirza Smajic, 220302230

Arda Berk Ozen, 210302084

Bakir Palalvra, 230302203

Imran Ali, 220302381

May 31st, 2026

International University of Sarajevo

Contents

1. Introduction

1.1 Project Overview

The IsaBegov Hamam Hotel Service Tracker is a web-based application developed for a small hotel to replace manual, paper-based staff coordination with a centralised digital system. Prior to this system, hotel staff relied on verbal handovers, paper notes, and informal messaging applications to communicate room status, tasks, and guest requests between shifts, which is a process prone to miscommunication and information loss.

The system provides hotel staff with four core operational capabilities: a task management board for daily jobs, a special requests section for shift-to-shift notes, a real-time room overview table, and a bookings calendar displaying upcoming reservations. It supports three user roles with different access levels: reception staff, cleaning staff, and hotel managers.

1.2 Project Scope

The scope of this project was defined in the Software Requirements Specification (SRS) produced during the requirements engineering phase. The application manages a single hotel with 15 rooms and a small number of staff users. It was designed specifically for internal use over a local network or internet connection, and is not intended as a general-purpose hospitality platform.

All features defined in the SRS were delivered in full. The system was built as an academic project, and the architecture was designed to be extensible for future additions such as push notifications, detailed reporting, and multi-property support.

2. Software Process and Development Methodology

2.1 Methodology

The team followed an Agile Scrum methodology, dividing the project into four two-week sprints with clearly defined goals, task assignments, and deliverables for each iteration. This approach was chosen for several reasons specific to the project context.

Because the team had varying levels of experience with the chosen technology stack (React, Node.js, MongoDB), an iterative approach allowed earlier sprints to focus on foundational learning and setup while later sprints built on a stable base. Agile also allowed the testing role

to run in parallel with development — test cases were written in Sprint 1 before any features existed, and testing occurred sprint-by-sprint as features were completed rather than as a single phase at the end.

2.2 Sprint Structure

Each sprint followed a consistent pattern. At the start of each sprint the team agreed on the API contracts between frontend and backend before any code was written, ensuring that both sides could develop in parallel without waiting on each other. A shared context document was maintained throughout the project and updated after each sprint so that all team members and any AI tools used for individual tasks had an accurate picture of the current system state.

Sprint 1 consisted of project setup, that is development of Node.js and Express backend, React frontend, MongoDB schema design, Git workflow, shared context document, Singleton DB connection, Postman collection, test case definitions (REQ1–REQ4), and seed data. Deliverables of Sprint 1 included working login/logout, MongoDB connected, React app running, all schemas defined, and Postman collection.

In sprint 2, Task management API and UI, Room overview API and UI, role-based access control, and RoomOperationsFacade skeleton were developed. Also, all previously delivered units of systems (authentication, tasks page, rooms page) were tested. Deliverables of sprint 2 included all task CRUD operations working, room status table with live updates, and role restrictions enforced.

In sprint 3 were developed additional features such as special requests API and UI, Bookings page, facade completion, and Manager Dashboard. Requests tests, Observer flow tests, Bookings tests, and end-to-end scenario tests were completed in this sprint. At the end of sprint 3 delivered features included all 5 modules working and auto task creation on room status change.

Lastly, in sprint 4 full SRS regression was tested against all requirements from SRS, presentation and final project report were prepared, and the complete system was deployed and running.

3. Requirements Engineering

3.1 Requirements Elicitation

Requirements were gathered through direct analysis of the hotel's operational workflow and communication with the hotel manager. The primary problem identified was the loss of information during shift handovers: tasks assigned during one shift were frequently not communicated to the next, and room statuses were tracked informally. Secondary problems included a lack of visibility into room occupancy and upcoming bookings across all staff roles.

The requirements process produced a Software Requirements Specification (SRS) document following the IEEE Std 830-1998 template. The SRS defined functional requirements across four system features, non-functional requirements covering performance, security, and usability, and a user class analysis identifying the three roles and their respective priorities.

3.2 Functional and Non-Functional Requirements

All functional requirements were delivered as specified in the SRS. The table below summarises the requirement groups and their implementation status.

ID	Feature	Description	Priority	Status
REQ1.1–1.6	User Authentication	Staff login/logout with username and password, session management, error messages, session persistence	High	**Delivered**
REQ2.1–2.7	Task Management	Create, view, update, and mark tasks complete; associate with rooms; prevent empty submissions	High	**Delivered**

REQ3.1–3.5	Special Requests / Notes	Add, view, and delete shift notes; auto-timestamp; associate with rooms; prevent empty submissions	High	**Delivered**
REQ4.1–4.5	Room Overview	Display all 15 rooms with status; authorised users update status; validate status values; reflect updates	Medium	**Delivered**
SRS 3.5	Booking Management	View upcoming reservations; track room occupancy based on bookings	Medium	**Delivered**
NFR — Performance	Response times	Page load < 3s; API response < 2s; support 5 concurrent users	High	**Delivered**
NFR — Security	Auth and data protection	Hashed passwords, HTTPS, role-based access, session termination, input validation	High	**Delivered**
NFR — Usability	Ease of use	Key tasks completable in 3–4 actions; meaningful error messages; mobile-friendly	High	**Delivered**

Table 1: Functional and Non-Functional Requirements

4. System Modeling

4.1 Use Case Diagram

The use case diagram defines three actors: Reception Staff, Cleaning Staff, and Hotel Manager. Each actor's use cases reflect their role in the hotel.

Reception staff interact with the full set of operational features: they log in and out, create and manage tasks, add and delete special requests, update room status, and view bookings. Cleaning staff have a narrower set of interactions focused on task execution: they view the task list, mark tasks as complete, read special requests, and view room status. The hotel manager has read-only visibility across all modules and exclusive access to the manager dashboard, which aggregates counts of rooms by status and bookings by occupancy state.

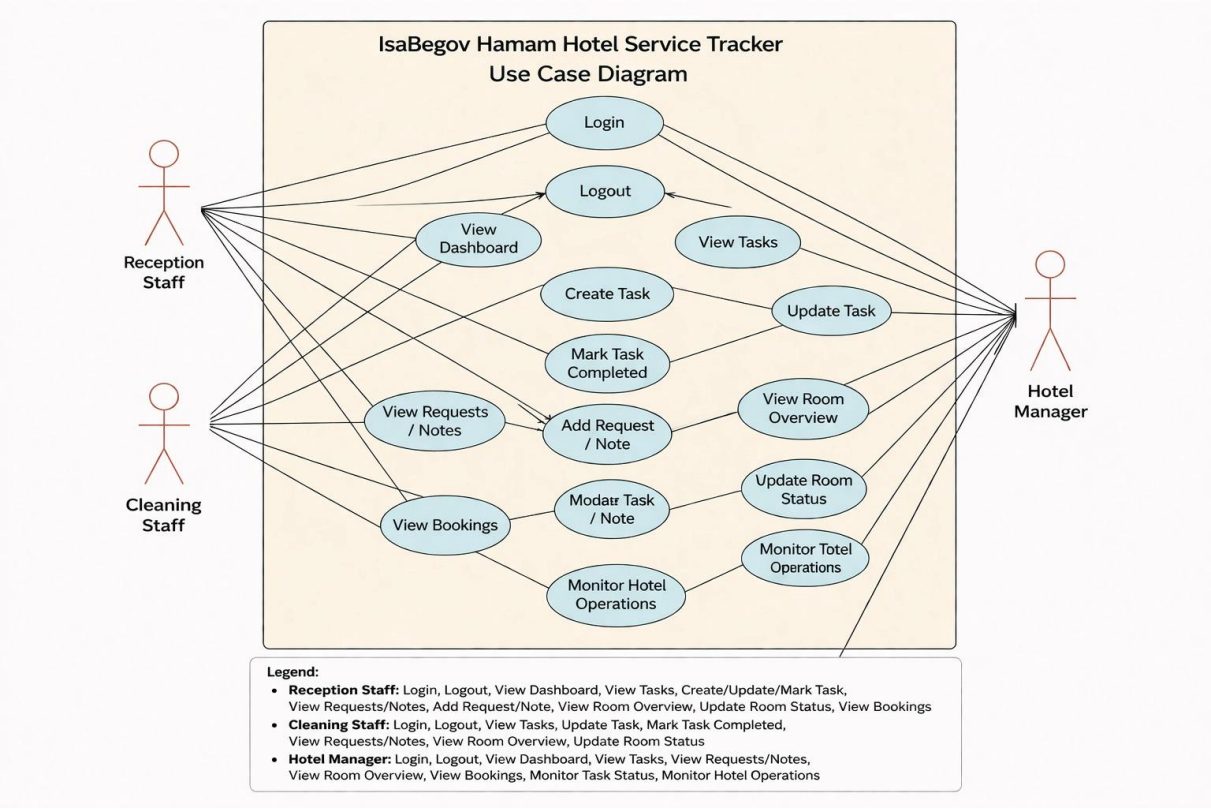

Figure 1: Use Case Diagram

4.2 Class Diagram

The class diagram captures both the domain model and the service layer. At the domain level, five entity classes map directly to MongoDB collections: User (with username, passwordHash, and role), Task (description, status, roomId, createdAt), Room (roomNumber, status), Request (note, roomId, createdAt, createdBy), and Booking (guestName, roomId, checkIn, checkOut, occupancyStatus).

The service layer introduces five service classes (AuthService, TaskService, RoomService, RequestService, BookingService), each owned by a single module following the Single Responsibility Principle. The class diagram also represents the three design patterns: DatabaseManager as a Singleton with a private constructor and static getInstance() method; RoomOperationsFacade as a class with associations to RoomService, TaskService, and RequestService; and the Observer relationship between RoomService as Publisher and TaskService and RequestService as Subscribers implementing the RoomObserver interface.

(3) CLASS DIAGRAM

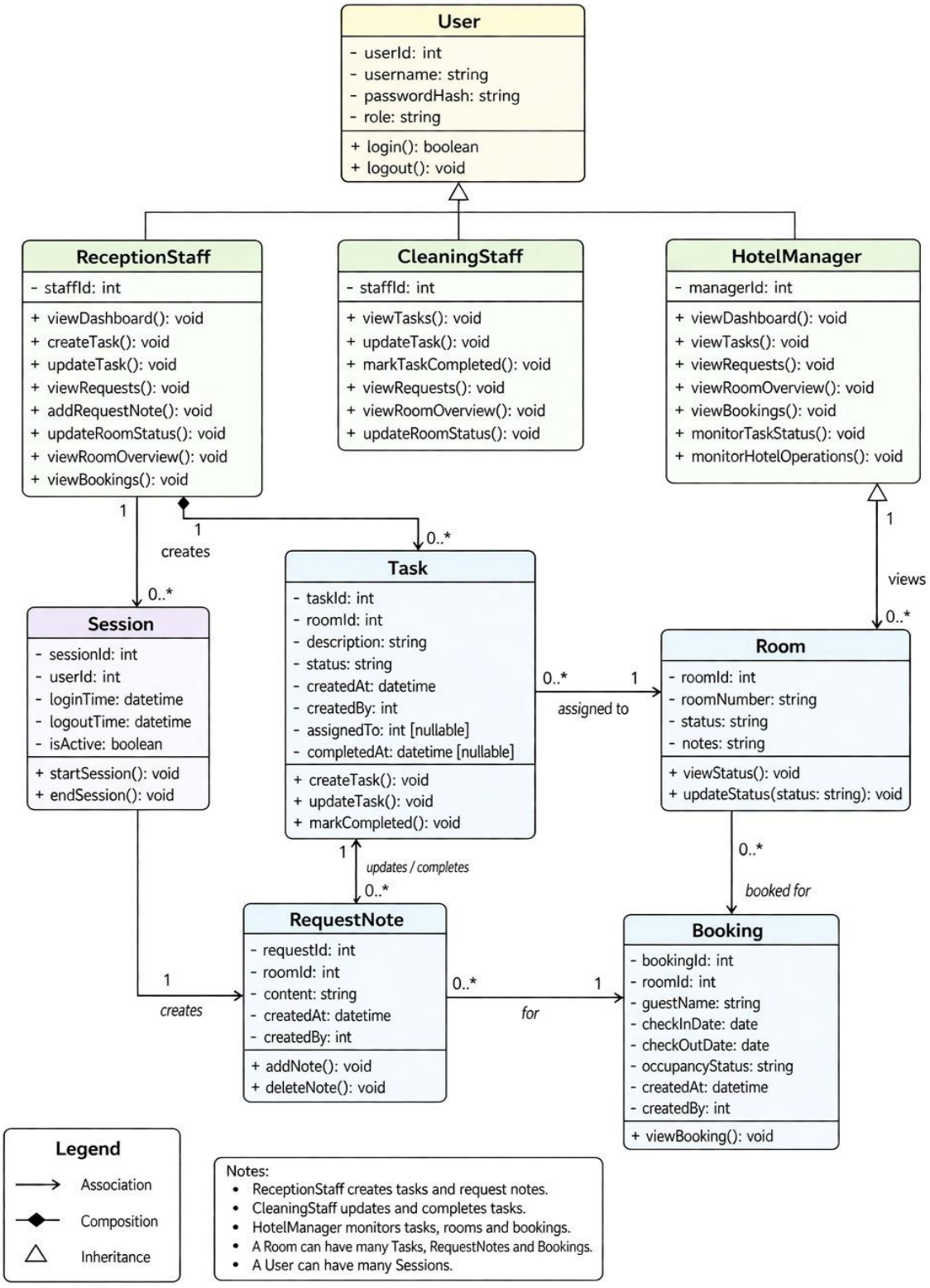

Figure 2: Class Diagram

4.3 Sequence Diagrams

Three key sequence diagrams were produced to capture most important runtime behaviours.

- Login flow - The user submits credentials through the React login page. The request reaches the Express AuthService, which queries MongoDB for the user record, compares the provided password against the stored bcrypt hash, creates a session on success, and returns the user role to the frontend. On failure, a generic error message is returned without specifying which field was incorrect.
- Room status update with Observer - Reception staff changes a room status to "needs-cleaning" through the room overview dropdown. The React frontend calls PUT /api/rooms/:id/status. The RoomOperationsFacade receives the request, calls RoomService to persist the status change, then triggers the Observer notification. TaskService receives the event and auto-creates a cleaning task for that room. RequestService receives the same event and logs a handover note with a timestamp. All three updates occur within a single API call from the frontend.
- Task completion - Cleaning staff marks a task as complete via the task list UI. The frontend sends PATCH /api/tasks/:id/complete to the TaskService, which updates the task status to "completed" in MongoDB. The frontend receives confirmation and removes the task from the active list immediately.

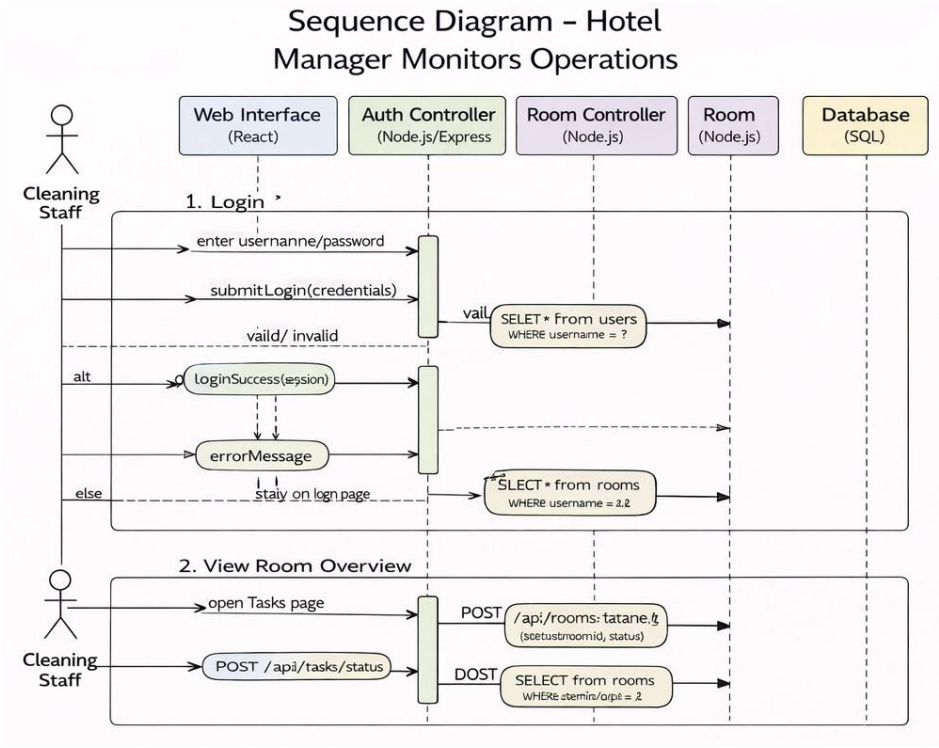

Figure 3: Sequence Diagram (Cleaning Staff)

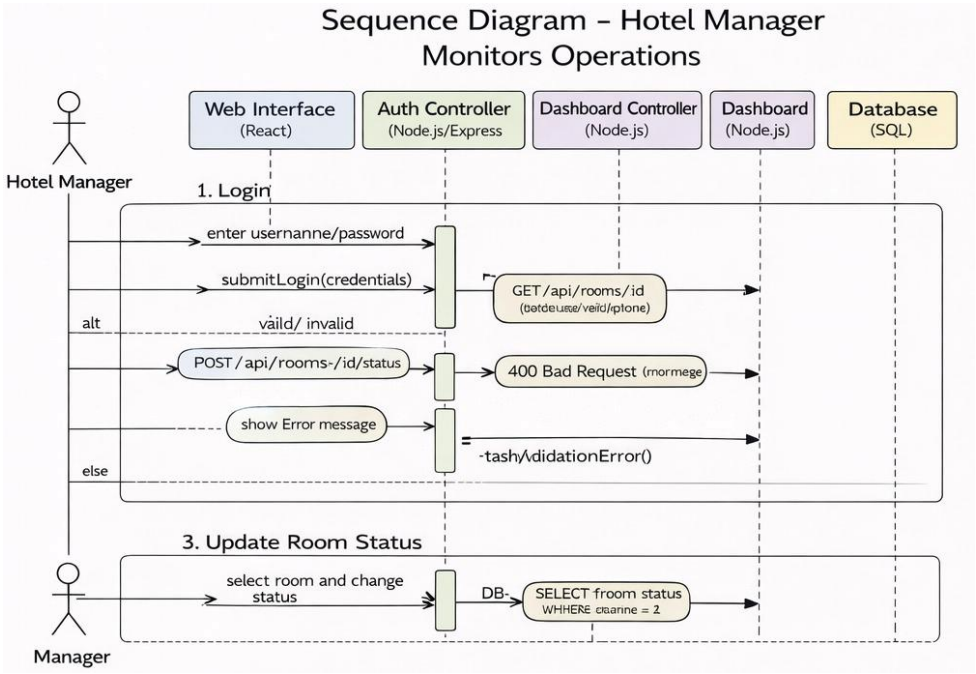

Figure 4: Sequence Diagram (Hotel Manager)

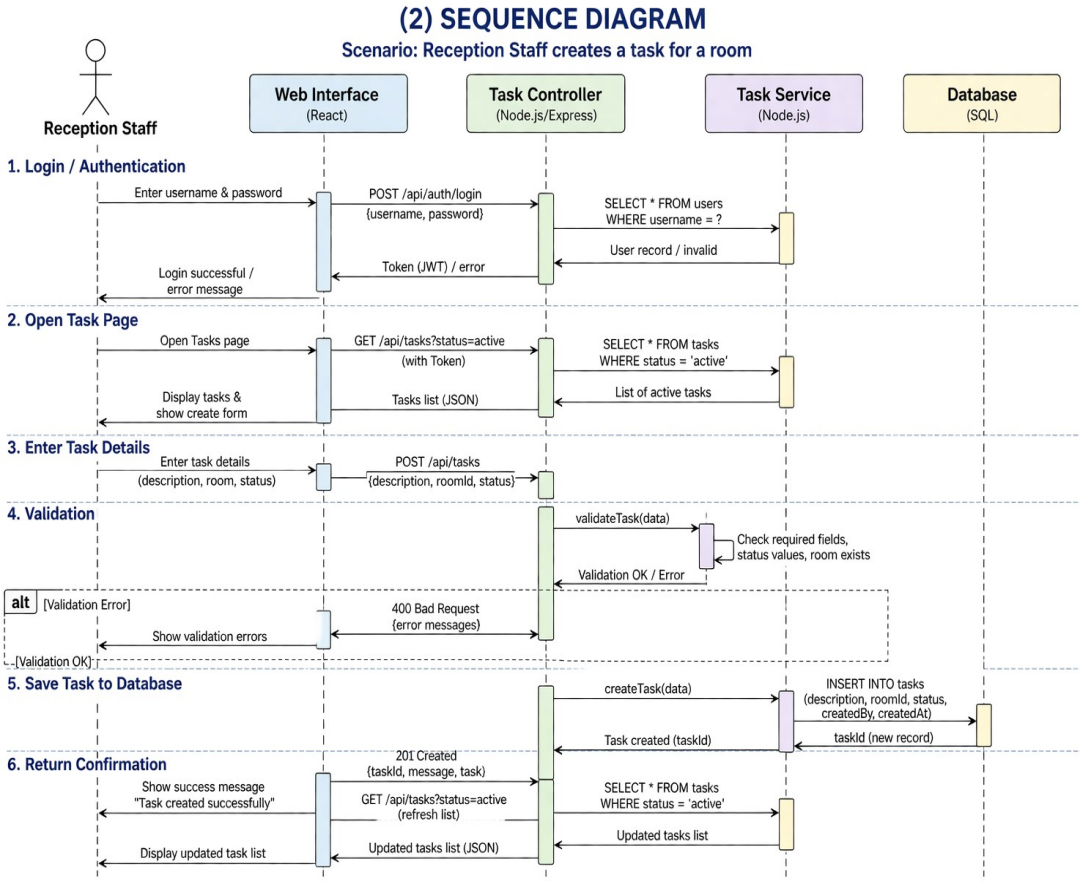

Figure 5: Sequence Diagram (Reception Staff)

5. Design and Implementation

5.1 Design Principles

The system design was guided by five core principles drawn from the course material, each applied concretely to the project.

- Abstraction - The React frontend communicates with the backend exclusively through the REST API. The frontend has no knowledge of how data is stored, validated, or processed, but it receives and sends JSON. Each module exposes a clean endpoint contract that hides all implementation details.
- Modularity - The backend is divided into five independent service modules (Auth, Tasks, Rooms, Requests, Bookings), each with its own route file, service class, and Mongoose model. Modules can be modified or tested independently without affecting others.
- Information hiding - Password hashing, session token management, and database schema details are never exposed to the client. The role-based middleware determines what each user type can see and do, and this logic is enforced server-side regardless of what the frontend requests.
- Cohesion - Each service class has exactly one responsibility. TaskService manages tasks only. RoomService manages room status only. The Single Responsibility Principle is enforced at the class level.
- Low coupling - Modules communicate through the Observer interface and the Facade rather than importing each other directly. The frontend is decoupled from backend implementation by the API contract. Adding a new subscriber to the Observer (such as a future notification service) requires no changes to existing code.

5.2 Module Implementation

The system is divided into five cohesive modules: AuthService, TaskService, RequestService, RoomService, and BookingService. Each of these modules has a single domain of responsibility.

Auth Module is responsible for login / logout, session management, password hashing, and role enforcement. Task Module is used to create / edit tasks, associate with rooms, mark as complete, and filter by status. Role of Request Module is to add shift notes, track timestamps,

link to rooms, and delete requests. Room Module is concerend with room status table, updating status, restricting invalid values, and real-time refresh. Lastly, Booking Module gives opportunity to view reservations, track occupancy, upcoming bookings, and room linking.

5.3 Frontend Implementation

The frontend is a single-page application built with React and Vite. React Router handles client-side navigation between the five pages: Login, Tasks, Rooms, Requests, and Bookings, with an additional Dashboard page visible only to managers.

Authentication state is managed by a custom useAuth hook backed by an AuthProvider component that wraps the entire application. The hook exposes user, role, isAuthenticated, login, and logout to any component via AuthContext. The user object is persisted to localStorage under the ibh_user key so that page refreshes do not require re-authentication.

The rooms page implements optimistic UI updates: when a staff member changes a room status via the dropdown, the status badge updates immediately in the UI before the PUT request to the server completes. If the request fails, the dropdown reverts to its previous value and an inline error message is shown on that row. This pattern provides a responsive feel without sacrificing data accuracy on failure.

The ConfirmModal component is used consistently across all delete operations, satisfying the SRS safety requirement that critical data cannot be accidentally removed without explicit confirmation.

6. Architectural Design

The system follows a three-tier layered architecture: Presentation (React frontend), Business Logic (Node.js and Express backend), and Data Access (MongoDB via Mongoose). Each layer communicates only with the layer directly below it. The React frontend never queries the database directly. It calls Express API endpoints. Express routes call service classes, and service classes interact with MongoDB through Mongoose models.

This architecture was chosen because it directly maps to the client-server structure described in the SRS, enforces separation of concerns at a structural level, and makes each tier independently replaceable. The presentation tier could be replaced with a mobile native app,

or the data tier could be migrated to a relational database, without requiring changes to the other tiers.

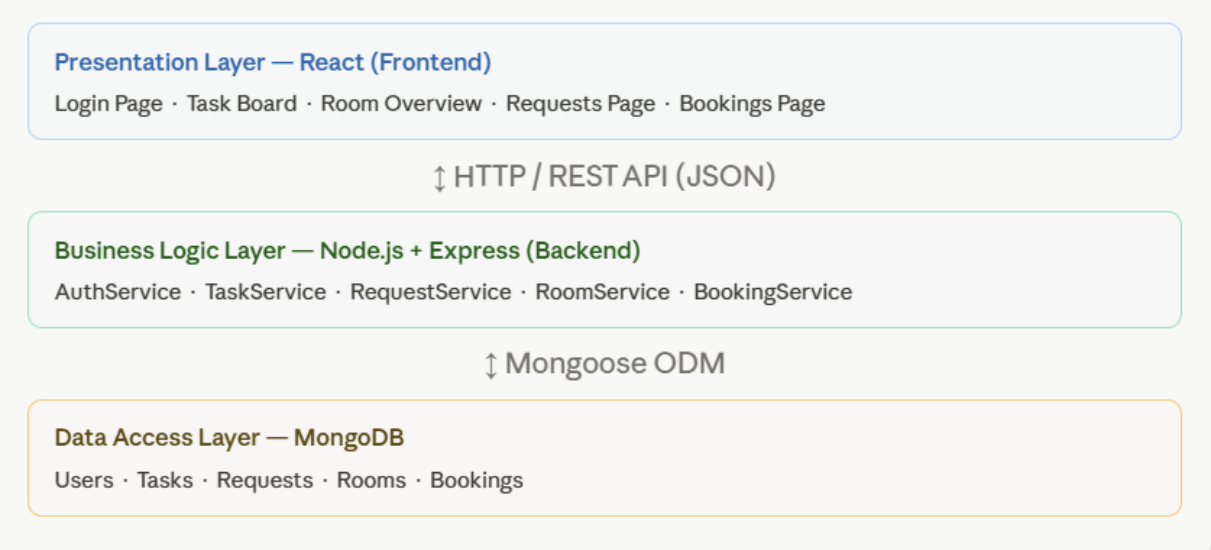

Figure 6: Layered Architecture of the System

6.1 Design Patterns

After careful consideration of all creational, structural, and behavioral patterns, three design patterns have been selected for this project based on direct fit with the system's needs:

- Singleton (creational) - Singleton pattern is applied in database connection instance, as well as in authentication session manager. This pattern is used because only one MongoDB connection should exist throughout the app lifecycle. Similarly, the session manager should be a single shared instance across all routes to maintain consistent state.
- Facade (structural) - Facade provides a single entry point for all room-related actions through RoomOperationsFacade method. When reception marks a room as "needs cleaning," the system must update room status, create a cleaning task, and optionally add a note. The Facade hides this multi-step process behind a single method call, reducing coupling between the frontend and backend subsystems.
- Observer (behavioral) - Observer pattern is applied when room status changes notify both the Task module and Request module automatically. That is, when a room's status changes other parts of the system such as task lists and shift notes need to react. The Observer pattern enables this without tight coupling between the Room module and the Task module.

6.2 Component Interaction

The diagram below shows the example of how the three layers and five modules interact at runtime for a typical user action such as a receptionist marking a room as "needs cleaning."

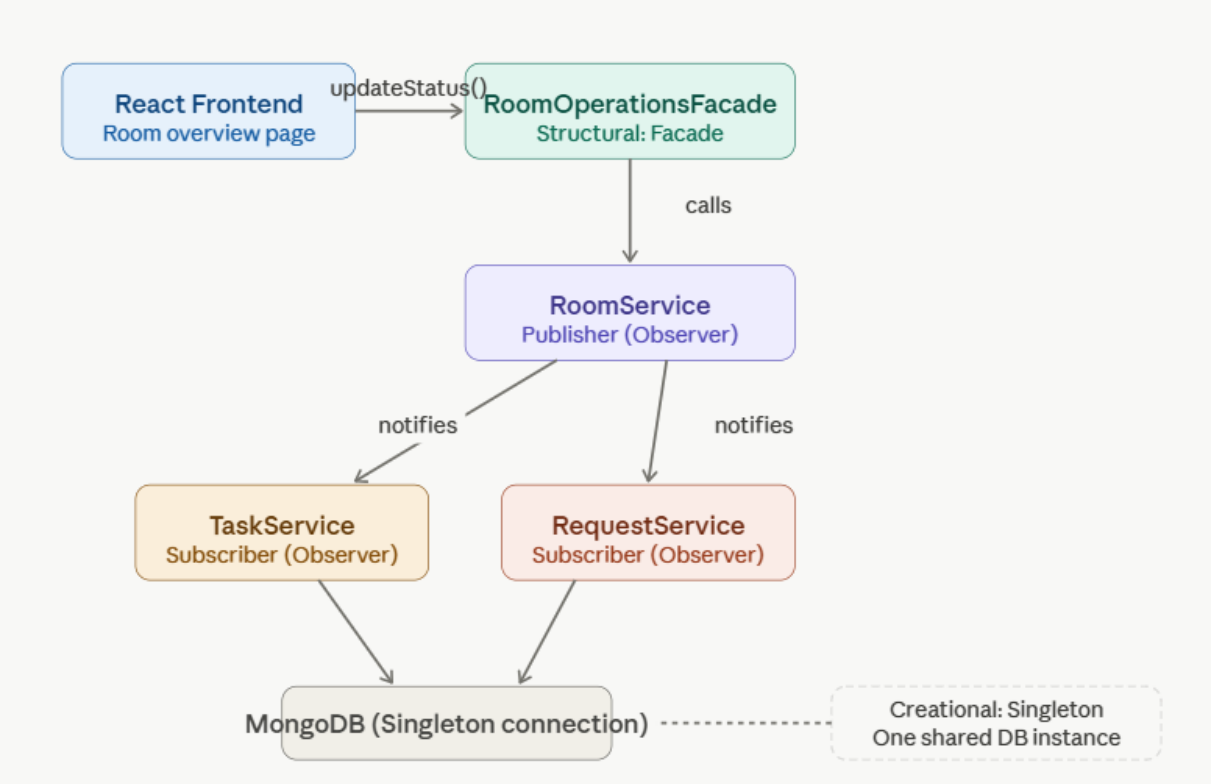

Figure 7: Component Interaction Diagram

6.4 Technology Stack

Technology stack used for each component of this system consists of:

- Frontend: React 18 with Vite, React Router, plain CSS. Component tests written with Vitest and React Testing Library.
- Backend: Node.js with Express. Session management via express-session. Password hashing via bcrypt. Input validation applied in route middleware.
- Database: MongoDB hosted on MongoDB Atlas (cloud). Mongoose ODM for schema definition and query abstraction. Collections: users, tasks, rooms, requests, bookings.
- Communication: REST API over HTTPS. All requests and responses use JSON. The agreed response envelope is { success: boolean, data: any, error?: string }.

7. Testing

Testing was integrated into the development process from Sprint 1 rather than deferred to the end of the project. Test cases for all SRS requirements were written in Sprint 1 before any features were implemented, creating a clear acceptance criterion for each piece of work. As features were merged in subsequent sprints, the corresponding test cases were executed immediately.

Two complementary testing approaches were used: automated unit and component tests for the frontend, and manual API testing via Postman for the backend. An end-to-end manual scenario was added in Sprint 3 to verify the complete shift handover workflow.

7.1 Testing Tools and Coverage

Table below shows the test tools used as well as area they cover.

Tool	Type	Coverage	Scope
Vitest + React Testing Library	Frontend automated	72 unit/component tests across 10 files	LoginPage, useAuth, NavBar, RoomsPage, RoomRow, StatusBadge, ConfirmModal, BookingsPage, BookingRow, DashboardPage
Postman Collection	Backend API manual	13 requests across 5 module folders, 4 assertions each	Auth, Tasks, Rooms, Requests, Bookings - all happy-path scenarios
Manual browser testing	Integration and UI	Full test case suite	Auth flow, task CRUD, room status, notes, bookings, role restrictions
End-to-end scenario	System-level	1 full shift handover scenario, 10 steps	Login → room checkout → auto task → auto note → cleaning → room available → manager view

Table 2: Testing Tools and Coverage

7.2 Automated Frontend Tests

Seventy-two automated tests were written across ten test files using Vitest and React Testing Library. Tests are organised into describe blocks by component, each grouping related behaviours. For example, "RoomsPage - initial load", "RoomsPage - optimistic status update", and "RoomsPage - retry" are three separate describe blocks within a single file.

All network calls are intercepted at the global.fetch level rather than using a mock server, keeping tests fast and isolated from the backend. Components that require authentication context receive it through direct AuthContext.Provider injection rather than the full AuthProvider, giving precise per-test control over the user role and avoiding unintended API calls inside the provider.

Negative test cases specifically cover: 401 responses showing masked error messages on the login page, empty arrays returning empty-state UI on list pages, network errors producing error banners with retry buttons, failed PUT requests triggering optimistic rollback on the rooms page, and missing optional props (such as an undefined room reference on a BookingRow) rendering graceful fallbacks.

7.3 Postman API Tests

The Postman collection "IsaBegov Hammam Testing" contains thirteen requests organised into five folders matching the five API modules. Every request carries a standard four-assertion Post-response script: HTTP 200 status, presence of the success field, success === true, and response time under 2000ms. The login request additionally asserts that data.role is present in the response.

The collection uses a baseUrl collection variable, allowing the entire collection to be redirected to a different environment (local development or production) by changing a single value. The collection file is stored in the repository under /postman/ and can be imported directly into Postman by any team member.

8. Project Management and Planning

The project was planned at two levels: a high-level sprint roadmap defined at the start of the project allocating modules to sprints and roles to team members, and a sprint-level task breakdown agreed at the beginning of each sprint. The high-level plan assigned one primary module per sprint, ordering features by SRS priority: authentication and setup first (Sprint 1),

the two high-priority modules Tasks and Rooms second (Sprint 2), the remaining features and design patterns third (Sprint 3), and polish, security, and deployment last (Sprint 4).

8.1 Task Assignment

Work was divided along two parallel tracks per sprint: backend API development and frontend UI development. For each feature, the backend and frontend developers agreed on the API contract at the start of the sprint before writing any code. This allowed both tracks to proceed simultaneously without blocking on the other.

The QA role ran continuously across all sprints rather than being concentrated at the end. Test cases were written in Sprint 1, backend tests were executed as endpoints were merged in Sprint 2, and frontend tests were written by the developer who built each component.

8.2 Risk Management

The main risks identified at project start were technology inexperience, time constraints within the academic semester, and coordination overhead in a five-person team. Several mitigations were applied.

Firstly, a shared context document was maintained throughout the project encoding the full system architecture, agreed API contracts, data models, and coding conventions. This document was pasted at the start of any AI-assisted work session to ensure consistent, project-aware assistance regardless of which team member was working.

Additionaly, the API contract agreement at the start of each sprint prevented the most common integration failure mode - mismatched request/response shapes between frontend and backend. The Postman collection served as an executable version of the API contract that could be run at any time to verify compliance.

9. Software Evolution and Maintenance

The architecture of this system was designed for extensibility at several points. The Observer pattern means that new event-driven features can subscribe to room status changes without modifying the RoomService publisher or any existing subscriber. A future push notification service, for example, could be added by implementing the RoomObserver interface and registering with RoomService — no existing code would need to change.

The Facade pattern means that complex multi-step operations can be extended without the frontend needing to know. If the checkout flow later needs to also notify a housekeeping supervisor or update a billing record, those steps are added to RoomOperationsFacade only, and the single frontend API call continues to work unchanged.

The modular service architecture means that individual modules can be replaced or upgraded independently. Migrating from MongoDB to a relational database, for instance, would require changes only to the Mongoose model files and the DatabaseManager Singleton — the service class interfaces would remain unchanged.

9.1 Planned Future Enhancements

The current implementation requires a page refresh to see changes made by other staff members. Adding WebSocket support (via Socket.io) would allow the room overview and task list to update in real time across all connected clients. An Observer subscriber could also be added to send push notifications to cleaning staff mobile devices when a new cleaning task is auto-created, eliminating the need to actively check the task list.

Another possible enhancement is reporting dashboard. The manager dashboard currently shows live counts. A reporting module could aggregate historical data (task completion rates, average cleaning times, room occupancy trends) for operational analysis.

10. Considerations

10.1 Security Considerations

Several security decisions were made that have implications for the system's long-term maintenance. Session-based authentication was chosen over JWT tokens because the system operates within a controlled hotel network with a small, known user base. Sessions are stored server-side and invalidated immediately on logout, which provides stronger security guarantees than stateless tokens in this context.

Role-based access control is enforced server-side in middleware, not client-side in the React application. UI elements may be hidden based on role, but all API endpoints independently verify the user's role before processing any request. This prevents privilege escalation through direct API calls.

10.2 Scalability Considerations

The system was designed and sized for a single small hotel with approximately fifteen rooms and ten staff users. If the system were to be expanded to larger properties or multiple simultaneous users, several architectural changes would be warranted.

The session-based authentication would need to be replaced with a shared session store (such as Redis) to support horizontal scaling across multiple server instances. The current in-process Observer pattern would need to be replaced with a message queue to support distributed event notification.

11. Conclusion

The IsaBegov Hamam Hotel Service Tracker was delivered in full within the planned four-sprint timeline. All functional requirements defined in the SRS were implemented: user authentication with role-based access control, task management with full CRUD operations, special requests with shift-to-shift note passing, a real-time room overview with status management, and a bookings tracking. All non-functional requirements for performance, security, and usability were also met.

The project successfully applied the design principles and patterns studied. The three-tier layered architecture cleanly separates presentation, business logic, and data concerns. The Singleton pattern ensures a single shared database connection, the Facade pattern simplifies complex multi-step room operations into a single API call, and the Observer pattern enables the automatic creation of cleaning tasks and handover notes without the frontend needing to orchestrate multiple requests.

Testing was integrated throughout the project lifecycle rather than deferred to the final sprint. Automated frontend tests, Postman API tests covering all endpoints, and end-to-end scenario all contributed to a high level of confidence in the delivered system.

The most significant technical challenge was coordinating a five-person team building a system with tightly coupled layers in a short timeframe. The API contract agreement at the start of each sprint, the shared context document, and the parallel QA track proved to be the most effective process practices.

The system architecture provides clear extension points for future development, allowing the system to grow with the hotel's operational needs without requiring a redesign.